

# CaVulner: Automated Context-Aware Identification of Vulnerable Versions

ANONYMOUS AUTHOR(S)

Modern software systems increasingly rely on third-party open-source components, which now constitute the majority of code in typical applications. While this reuse accelerates development, it also exposes systems to significant security risks, as known vulnerabilities in reused components can propagate downstream. To assess such risks, maintainers and Software Composition Analysis (SCA) tools depend on public vulnerability databases, such as the National Vulnerability Database (NVD) and GitHub Advisories, which specify affected version ranges. However, recent studies have shown that these databases often contain inaccurate or incomplete version metadata, leading to false negatives that leave systems unpatched or false positives that trigger unnecessary and potentially disruptive upgrades. This issue is amplified by the rapid growth in reported vulnerabilities and the adoption of large language models (LLMs), which accelerate vulnerability detection but shift pressure to the slower downstream tasks of confirmation and auditing.

In this paper, we propose *CaVulner*, an automated, LLM-assisted framework for context-aware identification of vulnerable versions. To localize vulnerable code, *CaVulner* first narrows candidate methods using commit change patterns and CVE/CWE metadata, then guides the LLM to extract the root cause through fine-grained reasoning over pre- and post-fix code. To infer affected versions, *CaVulner* traces vulnerability-inducing commits using git blame, identifies semantically duplicated commits via hunk hash comparison, and normalizes their associated tags into standardized version ranges. We evaluated *CaVulner* on 43 real-world Java open source projects that included 176 CVEs and 8,730 versions. The results show that *CaVulner* significantly outperforms state-of-the-art approaches, including V-SZZ, VERJava, and LLM4SZZ. It achieves an accuracy gain of at least 35.8% on CVE level. At the version level, F1-score increases by at least 23.94%, and the cost of using the LLM per vulnerability is only around \$0.06. It offers a scalable and semantically grounded solution for supply chain security.

## 1 Introduction

Modern software development heavily relies on third-party open-source software (OSS). The World Economic Forum highlights that OSS makes up as much as 70% to 90% of the code in software systems [10]. While the reuse of OSS significantly improves development efficiency and reduces costs, it also introduces substantial security risks. For instance, a report [6] by Black Duck reveals that 86% of codebases contain components with known vulnerabilities.

The number of known software vulnerabilities has been increasing rapidly in recent years, which extensively influenced the ecosystem. To manage this growing threat, organizations such as the National Vulnerability Database (NVD) [31] and GitHub Advisory Database [2] curate critical information on disclosed vulnerabilities, to support the wide usage of software supply chain tools like Software Composition Analysis (SCA), to assess potential risks. In this case, SCA tools rely heavily on the accuracy of vulnerability data, especially the correct vulnerable versions, for automated mappings of security risks in downstream user projects. However, multiple studies [9, 16, 33, 42] have shown that these vulnerability records often contain inconsistencies or errors, which can seriously mislead maintenance decisions. Moreover, as the integration of large language models (LLMs) has significantly accelerated the discovery of new vulnerabilities, the downstream tasks, like vulnerability confirmation and auditing, face increasing pressure to keep pace, and more automated solutions with less human effort, especially on vulnerable version identification, are urgently needed.

To precisely identify vulnerable versions in OSS, researchers have proposed a variety of solutions broadly categorized into dynamic analysis and static analysis. Dynamic approaches [26, 45, 47, 48] typically employ fuzzing to confirm whether a vulnerability can be triggered. While dynamic methods produce no false positives, they inherently suffer from false negatives due to the fundamental

limitations of fuzzing. Furthermore, establishing testing environments for every library proves impractical, imposing significant scalability constraints. Static analysis approaches primarily comprise version-controlled commit tracing [9, 14, 24, 32] and code similarity matching [28, 42, 49]. However, they heavily rely on the lines of changes in fix commits to trace the introduction of vulnerable code, while vulnerable code may not always be covered in fix commit diffs [36]. Furthermore, none of them analyze the nature of the vulnerability and the semantics of the code and are unable to distinguish between security fixes and non-security fixes.

Specifically, the SZZ algorithm based approaches are widely used in the identification of bug inducing commits (BIC) [14, 24, 32, 43], researchers also borrowed this algorithm for the identification of vulnerability inducing commits (VIC) [9]. However, vulnerabilities differ from bugs. While bugs are often introduced by recent commits, vulnerabilities are typically rooted in much earlier code. For example, Nikolaos et al. [5] found that the average lifetime of a vulnerability is approximately four years, indicating that vulnerabilities tend to remain latent for long periods before detection. Moreover, relying solely on patch content and commit messages is often insufficient for LLMs to fully capture the semantics of vulnerabilities. As a result, LLM4SZZ is unable to accurately identify vulnerable versions of OSS. The severity of these limitations was highlighted by the latest comprehensive empirical study [13] on this topic. After evaluating 12 state-of-the-art tools (including both tracing and matching-based methods) on over 1,000 vulnerabilities, the study revealed that no existing tool achieves an accuracy above 45.0%, largely due to their inability to handle complex patch patterns and lack of deep semantic understanding.

In summary, existing approaches face two key challenges. **Challenge 1:** How to distinguish security fixes from non-security fixes within a fix commit and accurately identify the vulnerability-introducing code. **Challenge 2:** How to effectively trace back to earlier vulnerable versions containing equivalent vulnerable code, given that traditional code similarity analysis and AST-based line mapping algorithms often fail due to code evolution over time.

To overcome these challenges, we propose a novel approach named *CaVulner*, which integrates multi-source context and leverages the capabilities of LLMs for vulnerability analysis and code understanding to identify vulnerable versions within the Java ecosystem. For **Challenge 1**, we develop a two-stage LLM-based module that leverages contextual cues while minimizing noise. The first stage focuses on coarse-grained localization by narrowing down candidate methods based on fix commit change patterns, such as whether a method contains only deletions, only additions, or both, and by incorporating relevant vulnerability information from CVE descriptions and CWE classifications. This guides the LLM to focus on methods most likely to contain the root cause. The second stage performs fine-grained reasoning by prompting the LLM to analyze semantic differences between the pre- and post-fix code within the selected method to identify the precise code fragment responsible for the vulnerability. For **Challenge 2**, we trace the origin of each vulnerable code block using a combination of git blame and LLM reasoning. We handle complexities such as cherry-picks, merges, and duplicated commits by computing hunk hashes to detect semantically identical changes across different commit hashes. All tags pointing to the inducing commits are collected, and the difference between these and the fix commit tags is computed. The resulting tag set is then normalized into standardized version ranges, representing the vulnerable versions.

Our experiments demonstrate the effectiveness of *CaVulner* on 176 real-world CVEs spanning 43 Java open-source projects, covering a total of 8,730 versions. Compared to state-of-the-art approaches, *CaVulner* improves at least 35.80% in accuracy at the CVE level. At the version level, precision increases by at least 10.40%, recall by 14.59% and F1-score by 23.94%. However, the cost of using the LLM remains low. On average, the LLM cost per vulnerability is only around \$0.06.

In summary, we make the following contributions.

- We propose a novel LLM-driven vulnerable version identification approach, *CaVulner*, which integrates multi-source context and leverages the capabilities of LLMs in vulnerability analysis and code understanding. The evaluation results demonstrate that *CaVulner* significantly outperforms state-of-the-art methods including V-SZZ, VERJava and LLM4SZZ. It achieves at least a 35.80% improvement in accuracy at the CVE level. At the version level, precision increases by at least 10.40%, recall by 14.59% and F1-score by 23.94%.
- We manually audited all 176 CVEs and identified the vulnerable versions, constructing a high-quality ground truth dataset containing 17,436 of these.
- We have released *CaVulner* and related experiment datasets on our website [1] to facilitate further research.

## 2 Preliminaries

In this section, we introduce the concepts of vulnerability-inducing commit, and a motivating example.

### 2.1 Vulnerability-Inducing Commit

The concept of Vulnerability-Inducing Commit (VIC) is conceptually derived from the well-established and widely studied notion of Bug-Inducing Commit (BIC) [39], which is formally defined as the commit in a version control system where a software defect is first introduced. Building on this foundational premise, a considerable number of researchers have dedicated efforts to proposing and refining improved techniques for identifying BICs, most notably through the development and refinement of various SZZ-based algorithms [14, 24, 32]. The original SZZ algorithm fundamentally operates by systematically tracing backward through the version control history starting from a known bug-fixing commit, with the objective of precisely identifying and isolating the specific earlier commits that were responsible for introducing the particular lines of code that were later modified or deleted in that fix, a process typically and effectively using built-in version control annotation or blame mechanisms. These subsequent variants of the SZZ methodology significantly enhance the original method by addressing common issues such as noise, inaccurate line mapping, and commit granularity.

Furthermore, with the recent and rapid emergence of powerful large language models, a growing body of recent studies has actively explored their potential integration into and enhancement of traditional SZZ-based workflows. For example, Tang et al. introduced LLM4SZZ [43], which leverages the semantic capabilities of language models to more accurately detect bug-inducing commits based on patch content and commit messages. By understanding the contextual meaning of code changes and commit descriptions, LLM4SZZ reduces false positives and identify extra commits that all baselines cannot.

Given the success of SZZ in the bug localization domain, it has subsequently been adapted for vulnerability analysis, particularly in identifying vulnerability-inducing commits and inferring vulnerability-affecting version ranges. One notable and direct adaptation of this methodology is exemplified by V-SZZ [9], which operates by attempting to systematically trace backwards from a known vulnerability-fixing commit in order to identify and pinpoint the specific earlier commit that originally introduced the security flaw, primarily through the analysis of lines of code that were deleted within the fix patch. However, this direct adaptation introduces significant limitations. Specifically, V-SZZ assumes all deleted lines are vulnerable and fails to address fix commits that only contain added lines.

More importantly, despite the shared conceptual basis, there are fundamental differences between bugs and vulnerabilities that are often not well accounted for in these approaches. For instance, while bugs are frequently introduced in recent commits and can often be detected within a short temporal

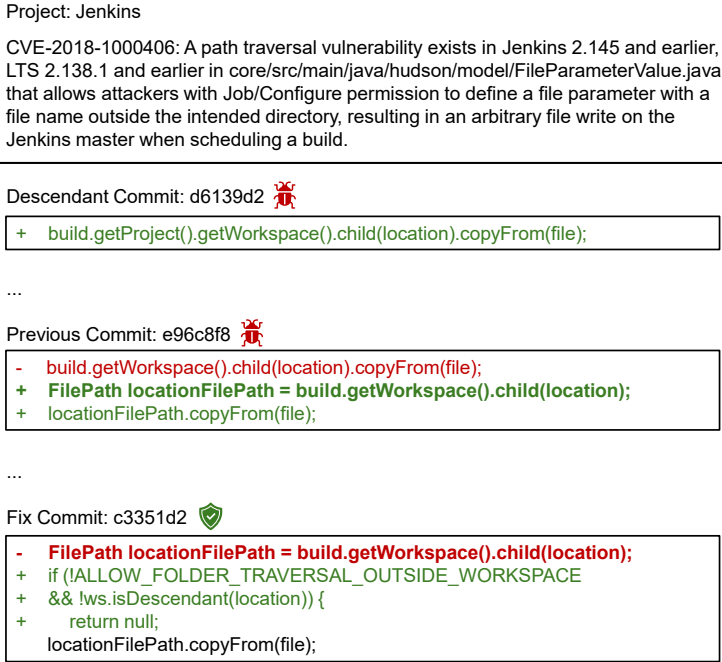


Fig. 1. The motivating example of CVE-2018-1000406

window, vulnerabilities tend to originate much earlier in the commit history, e.g., sometimes persisting for years before being discovered and fixed [5]. Moreover, vulnerabilities often arise from subtle semantic issues such as missing input validation or unsafe API usage, which are more difficult to detect using syntactic diff-based methods. Consequently, the effectiveness of existing SZZ-based approaches in vulnerability detection remains limited, as they fail to accommodate the unique characteristics of VICs.

This situation underscores the pressing and critical need for the development of more specialized techniques that are explicitly designed to model and address the distinct, often subtle, and exploit-oriented nature of security vulnerabilities, as opposed to merely adapting or directly borrowing from methodologies originally conceived for the localization of general, non-security-related software bugs.

## 2.2 Motivating Example

SZZ-based techniques have been widely adopted to infer vulnerable versions by identifying vulnerability-inducing commits. Among them, tools such as V-SZZ [9] represent state-of-the-art solutions in this space. However, their real-world effectiveness remains limited, particularly when applied to complex vulnerabilities [42]. To better understand the underlying reasons for these limitations, we conducted an in-depth case study to analyze the behaviors and failure modes of existing approaches.

For in-depth investigation, we include not only V-SZZ, but also VERJava [42], which leverages code similarity analysis to detect vulnerable versions, and LLM4SZZ [43], a recent method that utilizes large language models to identify bug-inducing commits. Although LLM4SZZ is designed

for bug detection rather than vulnerabilities, it represents the best-performing variant of SZZ to date and thus provides a strong reference point for evaluating the boundaries of current methodologies.

The motivating example is illustrated in Figure 1, which examines CVE-2018-1000406, an arbitrary file-write vulnerability caused by Jenkins's failure to properly normalize and constrain user input. The correct range of affected versions for this vulnerability is  $[1.312, 1.658] \cup [2.0, 2.145]$ . However, all three tools report the range as  $[1.362, 1.658] \cup [2.0, 2.145]$ , incorrectly identifying 1.362 as the lower bound. This discrepancy indicates a shared weakness across these tools in accurately tracing back to the true inducing commit.

Each tool failed for different reasons:

- **For V-SZZ**, it identifies vulnerability-inducing commits by treating every deleted line in the fix commit as vulnerable and tracing them back using the `git blame` command, aided by an AST-based line mapping to handle refactorings. In this case, it incorrectly attributes the commit `e96c8f8` (corresponding to version 1.362) as the inducing commit. However, deeper semantic inspection reveals that the actual vulnerable logic was introduced in a descendant commit of `e96c8f8`, which V-SZZ fails to reach due to it only tracing the vulnerable codes identified in the pre-patch commit, while neglecting the syntactic changes of these code during the backward tracing. Additionally, V-SZZ is unable to handle fixes that contain only added lines, rendering it ineffective in such scenarios.
- **For VERJava**, on the other hand, it uses code similarity to match vulnerable code patterns across versions. While this helps to detect structurally similar code, it struggles when semantic equivalence exists without syntactic similarity, particularly when code evolves significantly across versions. As a result, it fails to recognize vulnerable logic in earlier versions, leading to false negatives.
- **For LLM4SZZ**, it employs a large language model to analyze commit messages and code diffs to identify bug-inducing commits. Despite its improved performance for bugs, it still fails to identify the correct vulnerability-inducing commit in this case, again misattributing `e96c8f8` as the root cause. This failure highlights a key limitation: without contextual information such as CVE descriptions and a deeper understanding of both pre-fix and post-fix code, the model lacks the necessary reasoning to detect subtle security vulnerabilities. Moreover, like other SZZ-based approaches, LLM4SZZ is not equipped to perform extended historical tracing, which is often required to uncover long-lived vulnerabilities.

These findings collectively underscore the inadequacy of current approaches in reliably identifying vulnerability-inducing commits. Accurate vulnerable version identification demands not only precise syntactic tracing but also deep semantic understanding of code changes and security context.

### 3 Approach

To handle these limitations, we propose *CaVulner*, which integrates multi-source context and leverages the capabilities of LLMs in vulnerability analysis and code understanding to identify vulnerable versions.

As illustrated in Figure 2, our approach enhances vulnerable version identification by integrating large language models with semantic code analysis across three key steps. It first 1) identifies the vulnerable code by locating relevant functions and then 2) narrows down to fine-grained code blocks most likely responsible for the vulnerability, based on insights from multi-source context such as CVE descriptions and code context. Next, it traces these code blocks back to the specific commits that introduced the vulnerability, leveraging LLMs to reason about semantic changes that traditional syntax-based tools may miss. Finally, by analyzing both the inducing and fixing

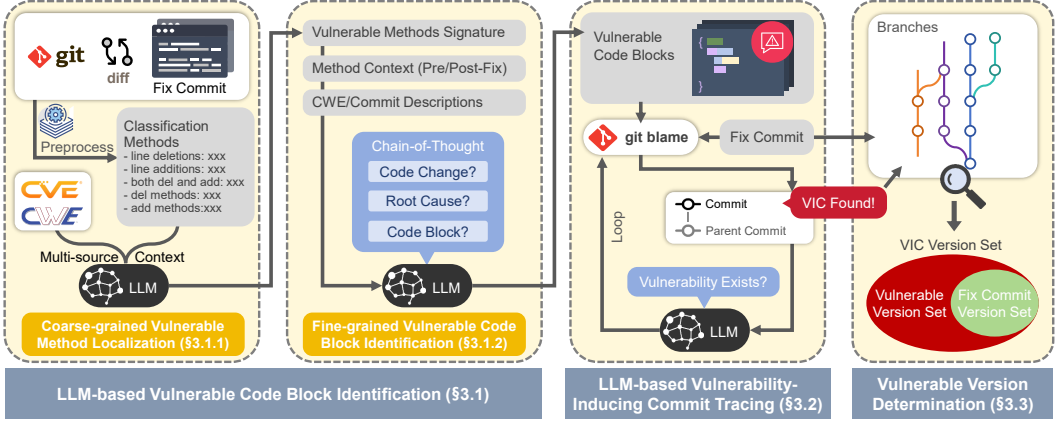


Fig. 2. Overview of CaVulner

commits, the approach determines the range of affected versions with improved accuracy, even in complex or long-lived vulnerability cases.

### 3.1 LLM-based Vulnerable Code Block Identification

The first step progressively narrows down the location of vulnerable code through two stages. It begins with coarse-grained method localization, identifying methods likely associated with the vulnerability based on contextual signals such as CVE descriptions, commit messages, and surrounding code. Building on this, the module then performs fine-grained analysis to isolate the specific code blocks within those methods that directly correspond to the vulnerable behavior. By combining these two stages, the module achieves both breadth and precision in identifying the code regions most relevant to the vulnerability.

**3.1.1 Coarse-grained Vulnerable Method Localization.** Directly feeding raw fix commit diffs to the LLM often leads to hallucinations because the model may struggle to distinguish the actual remediation logic from irrelevant code churn. To mitigate this issue, this submodule serves as a pre-filter to identify methods strictly relevant to the vulnerability and thereby reduces the search space for the subsequent fine-grained analysis. However, effectively filtering these methods presents a challenge as developers frequently interleave security patches with non-security maintenance such as refactoring, formatting, or feature updates.

To address this, we introduce Method Change Type (MCT) as a semantic abstraction mechanism. Unlike raw line-level diffs, MCT categorizes how a method is modified, establishing a correlation between the nature of the repair and the vulnerability type. For instance, fixing a *Missing Input Validation* vulnerability typically involves adding conditional checks, whereas remediating an *Information Exposure* vulnerability involves deletions. By explicitly defining these patterns, we guide the LLM to focus on methods whose modification signatures align with the expected repair strategy for the specific vulnerability.

Formally, we guide the LLM in localizing potentially vulnerable methods by a vulnerability knowledge tuple  $K_{\text{vul}} = (\text{MCT}, \text{CVE}, \text{CWE})$ , which is composed as follows:

- **MCT (Method Change Type).** MCT provides a high-level classification of modified methods in patches, facilitating the LLM's focus on vulnerable methods during Coarse-grained Vulnerable Method Localization. We define the following sets of methods.  $M_{\text{pre}}$ : The set of all methods in the pre-fix version of files modified by the commit.  $M_{\text{post}}$ : The set of all methods in the

post-fix version of files modified by the commit.  $M_{del\_lines}$ : The set of methods containing deleted lines of code from the fix commit (identified based on the pre-fix version).  $M_{add\_lines}$ : The set of methods containing added lines of code from the fix commit (identified based on the post-fix version). We classify method changes into five types based on these sets to reduce irrelevant information and enhance the LLM's focus. ① Methods with only line deletions:  $m \in (M_{pre} \cap M_{post} \cap M_{del\_lines}) \setminus M_{add\_lines}$ . ② Completely deleted methods:  $m \in M_{pre} \setminus M_{post}$ . ③ Methods with only line additions:  $m \in (M_{pre} \cap M_{post} \cap M_{add\_lines}) \setminus M_{del\_lines}$ . ④ Methods with both line additions and deletions:  $m \in M_{pre} \cap M_{post} \cap M_{del\_lines} \cap M_{add\_lines}$ . ⑤ Newly added methods:  $m \in M_{post} \setminus M_{pre}$ .

- **CVE (Common Vulnerabilities and Exposures)**. CVE is a standardized identifier for publicly disclosed security vulnerabilities. Each known security vulnerability is assigned a unique ID, and a standardized description is provided, detailing the characteristics of the vulnerability.
- **CWE (Common Weakness Enumeration)**. CWE is a community-maintained classification standard for software and hardware security weaknesses, aimed at systematically describing common weaknesses in code, design, or logic (e.g., root causes of vulnerabilities). It reveals the mechanism behind the vulnerability.

**Guided Prompt Construction.** We employ a guided prompting mechanism that integrates role assignment, multi-source context and an expert analysis framework to direct the LLM's reasoning.

- **Role Assignment.** The LLM is assigned the role of a senior Java vulnerability analysis expert and is tasked with performing logical inference based on the provided multi-source context.
- **Multi-source Context.** This consists of method change types, CVE ID, CVE description, CWE ID and CWE name.
- **Expert Analysis Framework.** We prompt the LLM to consider how different MCTs may be related to the remediation of vulnerabilities (e.g., line additions might indicate the introduction of new security checks). The model is guided to integrate multi-source data with its reasoning capabilities in order to identify the methods most associated with vulnerability and explain the rationale behind its conclusions.
- **Output.** The LLM outputs the signatures of the methods it has identified as vulnerable, which serve as part of the input to the subsequent fine-grained vulnerable code block identification module.

**3.1.2 Fine-grained Vulnerable Code Block Identification.** This submodule builds on the vulnerable candidate methods identified in the previous coarse-grained submodule by using the LLM to perform a fine-grained analysis. The goal is to identify vulnerable code blocks within these methods.

Specifically, for each vulnerable candidate method, we extract its complete code content before and after the fix and generate a diff between them. Compared to analyzing the full diff of a fix commit, which may contain much non-security content, this method-level organization significantly reduces the processing burden on the LLM and improves the accuracy of its analysis. Additionally, we provide the LLM with two supplementary inputs: the CWE description and the commit message for the fix. The CWE description highlights common vulnerability patterns and root causes, and the commit message explains the developer's intention behind the fix. Together, these inputs enable more accurate vulnerability analysis.

**Prompt Construction and LLM Reasoning Process.** We design prompts for the LLM that include the following information and guide it through a specific reasoning process.

- **Input Information.** This includes the complete source code of the vulnerable candidate method in both its pre-fix and post-fix versions, along with the corresponding code differences. Additionally, it contains the CWE description and the fix commit description.

---

**Algorithm 1** VIC Tracing
 

---

```

1: procedure TRACEVIC(fixCommit, vulnerableLine, cveInfo, cweInfo, methodSignature)
2:   queue  $\leftarrow$  empty queue
3:   prevCommit  $\leftarrow$  GitBlame(fixCommit, vulnerableLine)
4:   Enqueue(queue, prevCommit)
5:   while queue not empty do
6:     candidate  $\leftarrow$  Dequeue(queue)
7:     if not ExistsMethodInParent(candidate, methodSignature) then
8:       return candidate
9:     end if
10:    parentCommit  $\leftarrow$  ParentCommit(candidate)
11:    sourceCode  $\leftarrow$  GetMethodSource(parentCommit, methodSignature)
12:    (isVuln, newLines)  $\leftarrow$  LLMAnalyze(sourceCode, cveInfo, cweInfo)
13:    if not isVuln then
14:      return candidate
15:    else
16:      for all line in newLines do
17:        prevCommit2  $\leftarrow$  GitBlame(parentCommit, line)
18:        Enqueue(queue, prevCommit2)
19:      end for
20:    end if
21:  end while
22: end procedure

```

---

- **LLM Reasoning.** The LLM starts by analyzing the method-level code changes to understand how these modifications address the vulnerability. By integrating the common vulnerability patterns from the CWE description, the developer's intent from the fix commit message, and the specific code changes, the LLM further infers the root cause of the vulnerability. It then identifies and highlights the precise code blocks in the pre-fix version that are responsible for the vulnerability, providing an explanation for why these blocks are considered vulnerable.
- **Structured Output.** The final analysis is returned in JSON format and contains the following information: project name, CVE ID, fix commit hash, vulnerable file path, method signature, and identified vulnerable code blocks.

For a small number of CVEs, the code fixes cannot be explicitly attributed to any specific method. In such cases, due to the lack of effective method-level context, we provide the LLM with code before and after fix, the CVE and the CWE information to help it identify potentially vulnerable code blocks.

### 3.2 LLM-based Vulnerability-Inducing Commit Tracing

This module aims to identify the vulnerability-inducing commit (VIC) by tracing commits based on the previously identified vulnerable code blocks. First, for each vulnerable code block detected in the prior stage, we locate its corresponding line numbers of vulnerable codes in the pre-fix version of the repository. Next, we leverage a combination of the `git blame` command and the analytical capabilities of the LLM to identify VICs.

Algorithm 1 details the systematic approach employed for VIC tracing. Initialized with the fix commit, vulnerable line, CVE information, CWE information and method signature, it utilizes `git`

blame to obtain the previous commit, establishing an initial VIC candidate (lines 1-4). To address potential inaccuracies arising from code evolution, an iterative verification mechanism is triggered to examine the candidate's parent commit.

This verification involves a branching logic: if the vulnerable method does not exist in the parent commit, it indicates that the current commit introduced the vulnerable method, triggering the process to terminate and identify the current commit as the VIC (lines 7-9). In cases where the vulnerable method exists, an LLM performs a semantic assessment to determine if the vulnerability remains active in the parent version, using the CVE and CWE information together with the source code of the method in the parent commit. It is important to highlight that the LLM input is carefully curated to exclude the fix commit message, because the parent commit's message is likely unrelated to the vulnerability. If the LLM determines that the method in the parent commit is not vulnerable, then the previous commit is confirmed as the VIC (lines 10-14). Otherwise, the LLM identifies the vulnerable lines in the commit. We then apply the `git blame` command and further LLM analysis to these lines to trace earlier commits (lines 15-20).

### 3.3 Vulnerable Version Determination

In the context of version control systems (e.g., GitHub), tags are commonly used to denote versions. Once the LLM identifies the vulnerability-inducing commit (VIC) corresponding to a given fix commit, we can determine the set of vulnerable tags by examining the tags that include the VIC but not the fix commit.

However, in the real world, repositories often maintain multiple active branches (e.g., main, release-1.0 and release-2.0), and developers often propagate both VICs and fix commits across these branches through merges, cherry-picks, or manual copying. These operations result in duplicate commits, which are commits that introduce the same changes but have different commit IDs. Failure to identify all instances of duplicate fix commits or VICs can result in inaccurate reporting of vulnerable tags, leading to an underestimation or overestimation of their scope.

**Duplicate Commit Identification.** We identify duplicate commits by comparing the content of their patches. Specifically, we compute a hash for each hunk in every commit. For a given commit, we identify all other commits that share the same hunk hashes. If the set of hunk hashes in the candidate commit is equal to, or a subset of, that of another commit, we regard them as duplicates. This enables us to track semantically equivalent changes across branches, ensuring a more comprehensive identification of commits that introduce or fix vulnerabilities.

**Vulnerable Tags Determination.** Given a CVE, along with its associated fix commits and VICs, we analyse the git tags in the project's repository to identify the vulnerable tags. For each fix commit and its duplicates, we collect all tags that contain these commits, labeled  $T_f$ . Similarly, for each VIC and its duplicates, we gather all the corresponding tags,  $T_i$ . The set of vulnerable tags  $T_v$  is then those tags that belong to  $T_i$  but not to  $T_f$ :

$$T_v = \{t \mid t \in T_i \wedge t \notin T_f\}$$

This approach enables accurate identification of affected tags, even in complex branching scenarios involving cherry-picks or manual copying.

**Version Normalization.** We normalize GitHub tags to address non-standard naming conventions (e.g., 2016-04-03\_UMLet\_v14.2 or STRUTS\_2\_0\_0) that do not follow semantic versioning. In real-world development workflows, developers typically retrieve precompiled libraries from the Maven Central Repository [4] rather than building them directly from GitHub source code. Therefore, we treat Maven Central as the authoritative source for version identifiers and align GitHub tags accordingly. Specifically, we design project-specific normalization strategies to map non-standard GitHub tags to their corresponding Maven Central versions (e.g., mapping 2016-04-03\_UMLet\_v14.2

Table 1. Number of projects, stars, CVEs, fix commits, and tags in our dataset

| Project          | Stars | CVEs | Fix commits | Tags  |
|------------------|-------|------|-------------|-------|
| jackson-databind | 3.7k  | 62   | 147         | 213   |
| liferay-portal   | 2.2k  | 18   | 19          | 233   |
| jenkins          | 25k   | 17   | 17          | 1,835 |
| struts           | 1.3k  | 14   | 20          | 175   |
| tomcat           | 8.1k  | 7    | 8           | 488   |
| spring-framework | 59.6k | 7    | 9           | 340   |
| spring-security  | 9.4k  | 3    | 3           | 348   |
| other            | -     | 48   | 50          | 5,098 |
| <b>Total</b>     | -     | 176  | 273         | 8,730 |

to 14.2, STRUTS\_2\_0\_0 to 2.0.0). For projects available in Maven Central, we compute the intersection between the normalized GitHub versions that are tagged as vulnerable and the full set of versions in Maven Central. This intersection forms the final list of vulnerable versions. For projects not published in Maven Central, the normalized GitHub tags themselves serve as vulnerable versions.

4 Evaluation

Our implementation leverages the LangChain framework [3] for systematic pipeline construction. To mitigate stochastic noise in LLM-generated responses and reduce logical bias, we set the temperature to zero for LLMs. Our approach is implemented in about 8,500 lines of Python code in total. All experiments are conducted on a machine equipped with an AMD EPYC 9654 CPU, 32 GB of RAM, and running Ubuntu 20.04. We leverage API calls to interact with LLM services. We systematically evaluate *CaVulner* by addressing the following three research questions (RQs):

**RQ1. Effectiveness Evaluation.** How accurate is *CaVulner* in identifying vulnerable versions? How does the performance of *CaVulner* compare to state-of-the-art approaches?

**RQ2. Ablation Study.** What is the contribution of each component of *CaVulner* to its overall effectiveness? How effective is *CaVulner* when applied with different LLMs?

**RQ3. Resource Consumption.** What are the costs associated with LLMs?

4.1 Dataset

**CVE Dataset.** We leveraged the dataset compiled by Ponta et al. [35], who manually collected and curated vulnerabilities in open-source software along with their corresponding fixing commits. This dataset was also utilized in the studies by V-SZZ [9] and VERJava [42]. The dataset of V-SZZ covered various types of vulnerabilities. VERJava selected the TOP6 projects with respect to the number of CVEs having available patches. Additionally, it included Liferay-portal, which contains the highest number of CVEs in NVD. The CVEs in the VERJava dataset span a broad period of each project’s lifecycle, represent various vulnerability types, and involve patches of varying sizes.

Our study specifically targeted CVEs from this collection that affect open-source projects hosted on GitHub. To construct our dataset, we merged the CVEs used by V-SZZ and VERJava, then excluded those whose vulnerable versions were not available on GitHub, such as tomcat 5.x.x or 6.x.x. As shown in Table 1, it resulted in a corpus of 176 CVEs across 43 distinct Java projects and encompassing all 8,730 versions, including popular projects such as jackson-databind [19], liferay-portal [29], jenkins [22], struts [7], tomcat [8], spring-framework [40] and spring-security [41]. We

Table 2. Comparison with baselines at the CVE level (GT: Ground Truth)

| Project          | GT  | V-SZZ |       |          | VERJava |       |          | LLM4SZZ |       |          | CaVulner |       |          |
|------------------|-----|-------|-------|----------|---------|-------|----------|---------|-------|----------|----------|-------|----------|
|                  |     | True  | False | Accuracy | True    | False | Accuracy | True    | False | Accuracy | True     | False | Accuracy |
| jackson-databind | 62  | 0     | 62    | 0.00%    | 0       | 62    | 0.00%    | 0       | 62    | 0.00%    | 42       | 20    | 67.74%   |
| liferay-portal   | 18  | 8     | 10    | 44.44%   | 1       | 17    | 5.56%    | 6       | 12    | 33.33%   | 10       | 8     | 55.56%   |
| jenkins          | 17  | 3     | 14    | 17.65%   | 4       | 13    | 23.53%   | 3       | 14    | 17.65%   | 12       | 5     | 70.59%   |
| struts           | 14  | 9     | 5     | 64.29%   | 9       | 5     | 64.29%   | 8       | 6     | 57.14%   | 13       | 1     | 92.86%   |
| tomcat           | 7   | 4     | 3     | 57.14%   | 3       | 4     | 42.86%   | 6       | 1     | 85.71%   | 6        | 1     | 85.71%   |
| spring-framework | 7   | 4     | 3     | 57.14%   | 3       | 4     | 42.86%   | 5       | 2     | 71.43%   | 6        | 1     | 85.71%   |
| spring-security  | 3   | 2     | 1     | 66.67%   | 1       | 2     | 33.33%   | 2       | 1     | 66.67%   | 3        | 0     | 100.00%  |
| other            | 48  | 36    | 12    | 75.00%   | 15      | 33    | 31.25%   | 29      | 19    | 60.42%   | 37       | 11    | 77.08%   |
| Total            | 176 | 66    | 110   | 37.50%   | 36      | 140   | 20.45%   | 59      | 117   | 33.52%   | 129      | 47    | 73.30%   |

further verified and refined the fix commit datasets for both V-SZZ and VERJava based on GitHub issues and official project websites. As a result, we obtained a total of 273 fix commits. For each CVE within a project, we analyze all tags to identify vulnerable versions.

**Ground Truth.** We found that the V-SZZ and VERJava datasets share 23 overlapping CVEs. However, for 18 of these, the ground truth of vulnerable versions differs between the two datasets. Both methods rely exclusively on syntax by examining only the lines modified in fix commits. V-SZZ classifies any version containing deleted lines as vulnerable, while VERJava determines vulnerable versions by comparing the code of each candidate version with both the deleted code and the added code to decide whether the vulnerability exists. In practice, such commits often include nonsecurity-related changes, such as feature enhancements or code refactoring, which can result in inaccurate identification of vulnerable versions. To address this issue, we performed a comprehensive manual validation of all 176 CVEs and identified a list of vulnerable versions for each.

Our annotation principle was to identify vulnerable code based on the fundamental nature and semantics of the vulnerability. Specifically, for each CVE, we first analyze the CVE description and its associated CWE information. Drawing on expert knowledge and publicly available vulnerability-related resources, we identify the root cause of the vulnerability. We then examine the code before and after the fix to determine the vulnerable code blocks. After that, we applied `git blame` to trace these vulnerable lines through the version history. For each traced version, we examined whether the previous version contained the vulnerability. If not, we consider the current version to be the vulnerability-inducing version. If it did, we continued to trace the newly identified vulnerable lines further back. This recursive process was repeated until we found a version in which the vulnerability was no longer present. We define the range of vulnerable versions as the versions that span from the first identified vulnerable version to the version where the vulnerability was fixed. In total, this process resulted in a high-quality ground truth that includes 17,436 vulnerable versions in 176 CVEs.

The entire manual analysis was conducted by a team consisting of one Ph.D. student with more than ten years of experience in Java vulnerability research, including the discovery of zero-day CVEs, and two master's students. Each analyst performed in-depth research using personal experience and publicly available resources, such as official advisories and technical blogs. We use Krippendorff's alpha [25] to measure the agreement among the three annotators. The Krippendorff's alpha among the three annotators is 0.78, indicating a substantial agreement. The three annotators resolved the disagreements by group discussion until consensus was reached. The entire ground truth construction process took about one month.

Table 3. Comparison with baselines at the version level (GT: Ground Truth)

| Project          | GT     | V-SZZ |       |       |           |         |          | VERJava |       |       |           |        |          |
|------------------|--------|-------|-------|-------|-----------|---------|----------|---------|-------|-------|-----------|--------|----------|
|                  |        | TP    | FP    | FN    | Precision | Recall  | F1-score | TP      | FP    | FN    | Precision | Recall | F1-score |
| jackson-databind | 7,884  | 690   | 3,524 | 7,194 | 16.37%    | 8.75%   | 11.41%   | 5,834   | 3,096 | 2,050 | 65.33%    | 74.00% | 69.39%   |
| liferay-portal   | 454    | 287   | 61    | 167   | 82.47%    | 63.22%  | 71.57%   | 206     | 675   | 248   | 23.38%    | 45.37% | 30.86%   |
| jenkins          | 6,639  | 6,105 | 4     | 534   | 99.93%    | 91.96%  | 95.78%   | 4,189   | 2,132 | 2,450 | 66.27%    | 63.10% | 64.65%   |
| struts           | 321    | 321   | 148   | 0     | 68.44%    | 100.00% | 81.27%   | 242     | 131   | 79    | 64.88%    | 75.39% | 69.74%   |
| tomcat           | 176    | 153   | 0     | 23    | 100.00%   | 86.93%  | 93.01%   | 148     | 514   | 28    | 22.36%    | 84.09% | 35.32%   |
| spring-framework | 155    | 132   | 9     | 23    | 93.62%    | 85.16%  | 89.19%   | 114     | 0     | 41    | 100.00%   | 73.55% | 84.76%   |
| spring-security  | 35     | 35    | 9     | 0     | 79.55%    | 100.00% | 88.61%   | 16      | 107   | 19    | 13.01%    | 45.71% | 20.25%   |
| other            | 1,772  | 1,206 | 27    | 566   | 97.81%    | 68.06%  | 80.27%   | 1,205   | 133   | 567   | 90.06%    | 68.00% | 77.49%   |
| Total            | 17,436 | 8,929 | 3,782 | 8,507 | 70.25%    | 51.21%  | 59.24%   | 11,954  | 6,788 | 5,482 | 63.78%    | 68.56% | 66.08%   |

| Project          | GT     | LLM4SZZ |       |       |           |         |          | CaVulner |     |       |           |         |          |
|------------------|--------|---------|-------|-------|-----------|---------|----------|----------|-----|-------|-----------|---------|----------|
|                  |        | TP      | FP    | FN    | Precision | Recall  | F1-score | TP       | FP  | FN    | Precision | Recall  | F1-score |
| jackson-databind | 7,884  | 198     | 518   | 7,686 | 27.65%    | 2.51%   | 4.60%    | 5,793    | 0   | 2,091 | 100.00%   | 73.48%  | 84.71%   |
| liferay-portal   | 454    | 212     | 24    | 242   | 89.83%    | 46.70%  | 61.45%   | 360      | 84  | 94    | 81.08%    | 79.30%  | 80.18%   |
| jenkins          | 6,639  | 6,055   | 181   | 584   | 97.10%    | 91.20%  | 94.06%   | 6,223    | 0   | 416   | 100.00%   | 93.73%  | 96.77%   |
| struts           | 321    | 280     | 106   | 41    | 72.54%    | 87.23%  | 79.21%   | 315      | 0   | 6     | 100.00%   | 98.13%  | 99.06%   |
| tomcat           | 176    | 176     | 275   | 0     | 39.02%    | 100.00% | 56.14%   | 176      | 2   | 0     | 98.88%    | 100.00% | 99.44%   |
| spring-framework | 155    | 155     | 32    | 0     | 82.89%    | 100.00% | 90.64%   | 151      | 0   | 4     | 100.00%   | 97.42%  | 98.69%   |
| spring-security  | 35     | 27      | 0     | 8     | 100.00%   | 77.14%  | 87.10%   | 35       | 0   | 0     | 100.00%   | 100.00% | 100.00%  |
| other            | 1,772  | 1,127   | 14    | 645   | 98.77%    | 63.60%  | 77.38%   | 1,445    | 189 | 327   | 88.43%    | 81.55%  | 84.85%   |
| Total            | 17,436 | 8,230   | 1,150 | 9,206 | 87.74%    | 47.20%  | 61.38%   | 14,498   | 275 | 2,938 | 98.14%    | 83.15%  | 90.02%   |

## 4.2 Evaluation Metrics

We adopt accuracy, precision, recall, and F1-score as our evaluation metrics. The metrics are computed from two perspectives: the CVE level (accuracy) and the version level (precision, recall, and F1-score). Specifically, at the CVE level, an identification is labeled True only if the set of vulnerable versions exactly matches the ground truth. All other CVEs are labeled False. At the version level, each identified version is individually compared against the ground truth. A version present in the ground truth is counted as a TP; a version not present is counted as an FP; and a version that exists in the ground truth but is not identified is counted as an FN. The evaluation metrics are calculated as follows:

$$\text{Accuracy} = \frac{|\text{True}|}{|\text{True}| + |\text{False}|} \quad (1)$$

$$\text{Precision} = \frac{|\text{TP}|}{|\text{TP}| + |\text{FP}|} \quad (2)$$

$$\text{Recall} = \frac{|\text{TP}|}{|\text{TP}| + |\text{FN}|} \quad (3)$$

$$\text{F1-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

For example, consider a CVE whose ground truth includes versions  $[v_1, v_2, v_3, v_4]$ , while the approach identifies  $[v_3, v_4, v_5]$ . At the CVE level, this results in True = 0, False = 1, resulting in an accuracy of 0.00%. At the version level, TP = 2, FP = 1, FN = 2. Accordingly, the precision is 66.67%, the recall is 50.00%, and the F1-score is 57.14%.

### 4.3 RQ1. Effectiveness Evaluation

We select three representative state-of-the-art methods for identifying vulnerable versions in the Java ecosystem as our baselines: V-SZZ [9], which is based on commit tracing; VERJava [42], which leverages code similarity; and LLM4SZZ [43], which utilizes LLMs for vulnerability versions identification.

V-SZZ traces all deleted lines in a fix commit back to their origin using `git blame`, treating them as vulnerability-inducing lines. To improve the accuracy of locating vulnerability-inducing commits (VICs), it also uses an AST-based line mapping algorithm. However, V-SZZ cannot handle cases where the fix commit contains no deleted lines, and the AST-based line mapping is not always accurate. VERJava addresses this limitation by supporting fix commits without deleted lines. It computes vulnerable versions at both the function level and the entire patch level, and then synthesizes the results to determine the final set of vulnerable versions. Since VERJava requires comparing every version, the computational cost is very high. As a result, the authors evaluated only a subset of versions. In contrast, to ensure the accuracy of our evaluation, we evaluate all versions of each project. For example, the VERJava study only assessed 13 versions of the Liferay Portal project, while we evaluated all 233 versions. LLM4SZZ employs rank-based identification and context-enhanced identification to handle different types of fixing commits.

Table 2 and 3 present the results of *CaVulner* and baselines in identifying vulnerable versions in the CVE level and version level, respectively. Both LLM4SZZ and *CaVulner* use DeepSeek V3 as the large language model. According to the evaluation metrics, the criteria for CVE level statistics are particularly strict: if even a single predicted vulnerable version does not match the ground truth, the result cannot be counted as true. As a result, the accuracy at the CVE level is generally lower than precision and recall at the version level.

Despite the stringent requirements for CVE-level accuracy, Table 2 reveals that *CaVulner* achieves the highest overall accuracy of 73.30%, outperforming LLM4SZZ (33.52%), V-SZZ (37.50%) and VERJava (20.45%). These results highlight the effectiveness of *CaVulner* in accurately identifying complete vulnerable version sets for each CVE.

We examined the 47 failure cases where *CaVulner* failed, aiming to clarify the underlying causes of these inaccuracies. Specifically, 8 cases erroneously included non-vulnerable early versions, while 39 failed to cover the complete vulnerable range. Further analysis reveals that 27 failures stemmed from misjudged semantic equivalence, while 14 cases resulted from errors in identifying duplicated commits, and the remaining 6 were caused by incorrect initial code localization.

At the version level, where predictions are evaluated at a finer granularity, Table 3 demonstrates that *CaVulner* again achieves superior performance across all metrics. It achieves the highest precision (98.14%), recall (83.15%), and F1-score (90.02%), indicating that it can reliably identify most vulnerable versions while introducing few false positives. In comparison, LLM4SZZ achieves a lower recall (47.20%) and F1-score (61.38%), suggesting its vulnerability identification is more conservative and incomplete. Meanwhile, VERJava, although showing slightly better recall (68.56%) than V-SZZ (51.21%), suffers from a high false positive rate, as reflected by its lower precision (63.78%) and F1-score (66.08%).

Notably, *CaVulner* achieves 100% precision in several projects such as *jackson-databind*, *jenkins*, *spring-security*, and *struts*, indicating its ability to avoid introducing any irrelevant versions during prediction. Even in large-scale projects like *jenkins* and *jackson-databind*, which contain thousands of versions, *CaVulner* maintains consistently high accuracy and recall.

Table 4. The results of different variants

|                      | CVE Level | Version Level |        |          |
|----------------------|-----------|---------------|--------|----------|
|                      | Accuracy  | Precision     | Recall | F1-score |
| <i>CaVulner_diff</i> | 49.43%    | 94.11%        | 58.12% | 71.86%   |
| <i>CaVulner_ast</i>  | 65.91%    | 99.22%        | 80.23% | 88.72%   |
| <i>CaVulner</i>      | 73.30%    | 98.14%        | 83.15% | 90.02%   |

RQ1: *CaVulner* consistently outperforms state-of-the-art baselines at both the CVE and version levels. The improvements in accuracy, precision, recall, and F1-score demonstrate the effectiveness of *CaVulner* in identifying vulnerable versions.

#### 4.4 RQ2. Ablation Study

In this section, we evaluate the effectiveness of the key components of *CaVulner*. During the vulnerable code identification stage of *CaVulner\_diff*, we remove the coarse-grained vulnerable methods localization and fine-grained vulnerable code blocks identification based on multi-source information fusion. Instead, we let the LLM identify the vulnerable code based on the fix commit diff files and CVE information. In the VIC tracing phase, we use the same approach as *CaVulner*. In *CaVulner\_ast*, after identifying the vulnerable code blocks, we replace the use of LLM for tracing the VICs with an AST-based line mapping approach.

As shown in Table 4, at the CVE level, *CaVulner\_diff* achieves only 49.43% accuracy, indicating that without Coarse-grained Vulnerable Method Localization and Fine-grained Vulnerable Code Block Identification the LLM often misidentifies vulnerable versions. Introducing AST-based tracing raises CVE accuracy to 65.91%. Finally, the full *CaVulner* reaches 73.30%, outperforming *CaVulner\_diff* and *CaVulner\_ast*.

To validate the effectiveness of our LLM-based VIC tracing module, we further compare its performance against the AST-based baseline (*CaVulner\_ast*) using the strict CVE-level accuracy metric. As shown in Table 4, the AST-based approach achieves a CVE accuracy of 65.91%. Since CVE-level accuracy requires an exact match of the entire vulnerable version range, any deviation in identifying the VIC results in a failure for that CVE. By replacing the AST algorithm with our LLM-based semantic tracing, *CaVulner* significantly improves the accuracy to 73.30%. This absolute gain of 7.39% demonstrates that the LLM-based approach is superior in correctly pinpointing the exact VIC. Unlike the AST method, which may be disrupted by complex code refactorings and fail to identify the true origin, the LLM's semantic understanding enables *CaVulner* to correctly determine the start of the vulnerability lifecycle in substantially more cases, thereby securing the correctness of the final version sets.

At the version level, *CaVulner\_diff* achieves a high precision (94.11%) but suffers from poor recall (58.12%) and a moderate F1-score (71.86%). *CaVulner\_ast* improves substantially on recall (80.23%) and F1-score (88.72%) and achieves high precision (99.22%). However, both its recall and F1-score remain lower than those of *CaVulner*. This demonstrates that although AST-based tracing can boost precision, integrating LLM-driven VIC tracing is essential to maximize recall and overall F1 performance.

We further evaluated three additional LLMs, including DeepSeek R1, GPT 4.1-mini, and Qwen turbo. Table 5 compares the performance of four different large language models on both CVE and version levels, all run with zero temperature to ensure determinism.

Table 5. The results of different LLMs

| LLM          | CVE Level | Version Level |        |          |
|--------------|-----------|---------------|--------|----------|
|              | Accuracy  | Precision     | Recall | F1-score |
| deepseek-v3  | 73.30%    | 98.14%        | 83.15% | 90.02%   |
| deepseek-r1  | 72.73%    | 97.63%        | 85.56% | 91.20%   |
| gpt-4.1-mini | 52.27%    | 87.78%        | 62.88% | 73.27%   |
| qwen-turbo   | 43.75%    | 84.52%        | 60.98% | 70.84%   |

Table 6. Average LLM resource consumption per CVE vulnerable version identification

| LLM          | input(k) | output(k) | total(k) | cost(\$) | time(s) |
|--------------|----------|-----------|----------|----------|---------|
| deepseek-v3  | 102.47   | 28.25     | 130.72   | 0.060    | 38.68   |
| deepseek-r1  | 100.83   | 296.53    | 397.36   | 0.721    | 206.49  |
| gpt-4.1-mini | 91.55    | 31.08     | 122.63   | 0.086    | 21.61   |
| qwen-turbo   | 92.86    | 22.67     | 115.53   | 0.006    | 21.27   |

Overall, the performance of different LLMs on the vulnerable version identification task varies, reflecting each model's inherent capacity for understanding and reasoning about vulnerabilities. DeepSeek-v3 and DeepSeek-R1 exhibit comparable performance, and both achieve the best results among all evaluated LLMs.

At the CVE level, every LLM outperforms the traditional baselines. Even the weakest model, Qwen-turbo, attains an accuracy of 43.75%, surpassing the best baseline (V-SZZ) at 37.50%. This demonstrates that, under the strict all-match criterion, general-purpose LLMs can still exceed baseline approaches.

At the version level, even the worst-performing Qwen-turbo still has an F1-score (70.84%) higher than the best F1-score in the baseline, VERJava (66.08%), further demonstrating the advantage of *CaVulner* in identifying vulnerable versions.

RQ2: The ablation study validates that each key component of *CaVulner* is essential to its effectiveness. Additionally, all evaluated LLMs outperformed the baseline in vulnerability version identification.

#### 4.5 RQ3. Resource Consumption

We implement *CaVulner* by invoking LLM APIs. Since analyzing the root cause of a vulnerability and localizing the corresponding code blocks requires a non-trivial amount of time and tokens, we assess the cost of LLM usage. As shown in Figure 2, *CaVulner* relies on LLM in three stages: Coarse-grained Vulnerable Methods Localization, Fine-grained Vulnerable Code Block Identification, and Vulnerability-inducing Commit Tracing. We evaluate the token consumption, and time overhead during these stages.

Table 6 shows the average LLM resource consumption per CVE vulnerability-version identification. In terms of input, output, and total tokens, DeepSeek V3, GPT 4.1-mini, and Qwen turbo exhibit similar consumption. DeepSeek R1, as a reasoning model, exhibits significantly higher output token consumption. It consumes about ten times more output tokens than DeepSeek V3

while using an equivalent number of input tokens. For runtime, GPT 4.1-mini and Qwen turbo require the least processing time, DeepSeek V3 takes approximately twice as long, and DeepSeek R1 incurs the highest latency.

**RQ3:** *CaVulner* demonstrates high economic feasibility. Using DeepSeek V3, the average cost is only \$0.06 per CVE. This confirms that standard LLMs offer a scalable and cost-effective solution for automated vulnerable version identification.

## 5 Discussion

**Threats to Validity.** In constructing the ground truth, we manually identified the vulnerable versions for each CVE, which may introduce potential inaccuracies. To mitigate this threat, three authors with cybersecurity expertise, including one Ph.D. student and two master's students, independently examined each CVE based on their domain knowledge and publicly available information. Discrepancies were resolved through collaborative discussion until consensus was reached. On average, six CVEs were processed per day, and the entire process took approximately one month. To capture the complexity of real-world vulnerabilities, our dataset incorporates diverse data qualities. Among the 176 selected CVEs, 58.6% contain low-quality commits with irrelevant messages, 9.9% involve complex fixes that modify more than five files, and 5.7% lack CWE information. Such heterogeneity suggests that *CaVulner* possesses a significant degree of tolerance toward low-quality input data.

**LLM Capability and Overhead.** Previous approaches to vulnerable version identification were limited in their ability to understand vulnerability semantics and code logic. The emergence of large language models (LLMs) has addressed this limitation by enabling more accurate semantic reasoning. However, LLMs are not always precise and may still produce incorrect results. As LLM technology continues to evolve, its effectiveness in vulnerability analysis is expected to improve further. Although LLM-based identification requires more time compared to traditional methods, the improvement in accuracy makes the additional computational cost acceptable.

**Other Ecosystems.** This study focuses on the Java ecosystem for three reasons. Java is the dominant language in enterprise-level applications, and its vulnerabilities can have far-reaching consequences. For example, Log4Shell vulnerability remains one of the most critical flaws ever discovered [15]. Moreover, the first author has worked extensively on Java vulnerability research, including zero-day discovery, and possesses deep practical expertise with Java codebases and patch processes. This experience substantially reduces annotation errors when constructing the ground truth of vulnerable versions. Finally, to ensure accurate and efficient extraction of method change patterns from fix commits, we employ a Java AST parsing library. Extending vulnerable-version identification to other ecosystems such as C, C++, and Python represents an important direction for future work.

## 6 Related Work

To precisely identify vulnerable versions in OSS, researchers have proposed a variety of solutions broadly categorized into dynamic analysis and static analysis.

### 6.1 Dynamic Analysis for Vulnerable Version Identification

Dynamic analysis approaches typically and fundamentally rely on the systematic application of fuzzing techniques to empirically confirm and demonstrate the concrete exploitability of a potential vulnerability by attempting to trigger its underlying abnormal program behavior under controlled execution conditions.

For example, SemFuzz [47] proposes and develops a methodology fundamentally rooted in program semantics, which is precisely architected for the automated synthesis of functional Proof-of-Concept (PoC) exploit code, then repurposes the synthetically generated PoCs as seed inputs to direct a subsequent, systematic fuzzing campaign, with the objective of triggering and thereby uncovering latent vulnerabilities embedded within the target software system. PatchFuzz [45] begins its operation by first identifying the precise Git commits that are documented as fixes for specific security flaws, then proceeds to extract the associated PoC for each fix, and finally synthesizes this collected information with the semantic and syntactic details of the patch diff to guide focused fuzzing on the modified code regions. TransferFuzz [26] adopts a trace-guided fuzzing approach wherein it meticulously records and archives the execution traces from a known vulnerable version of the program, subsequently reusing and adapting these historical traces to precisely steer and prioritize its fuzzing campaigns when targeting newer, patched, or derivative versions, thereby systematically determining whether the propagation of reused code components continues to expose the same underlying vulnerability. DeltaFuzz [48] performs an extraction of version-difference information and strategically utilizes this to prioritize and guide the generation of fuzz inputs along the repair boundary, with the ultimate objective of verifying whether subsequent releases or later versions of the software remain susceptible to the original security flaw.

In addition, Directed Greybox Fuzzing (DGF) plays a crucial role in vulnerable version identification by steering the fuzzer towards specific target sites, such as potentially vulnerable functions. AFLGo [11] integrates a distance-based metric into the energy scheduling of the fuzzer, prioritizing inputs that execute paths closer to the target vulnerability, thus facilitating the efficient reproduction of known CVEs across different versions. Building on this, Hawkeye [12] optimizes the direction mechanism by incorporating static analysis of the call graph and function-level similarity, addressing the issue of trace explosion in long-path vulnerability verification. More recently, WindRanger [17] introduces a deviation-based directed fuzzing approach specifically designed for patch analysis. It identifies deviation basic blocks between the vulnerable and patched versions and directs the fuzzer to explore these divergences, making it highly effective for verifying whether a specific version contains the unpatched logic. To further improve efficiency in large-scale systems, Beacon [20] utilizes static analysis to compute reachability preconditions and aggressively prunes execution paths that cannot reach the target vulnerability, thereby significantly accelerating the validation process. Similarly, SelectFuzz [30] enables selective fuzzing of specific code regions within complex binaries by isolating the execution of relevant functions, allowing for precise verification of vulnerabilities in targeted components without the overhead of whole-system execution.

While dynamic analysis methods produce no false positives, they inherently and unavoidably suffer from the problem of false negatives. This limitation is fundamentally rooted in the stochastic nature of fuzzing techniques, which often struggle to achieve sufficient code coverage or satisfy complex path constraints required to trigger deeply hidden vulnerabilities. Furthermore, establishing, configuring, and maintaining isolated and functionally complete testing environments for every distinct library version or dependency encountered in a large-scale system proves to be impractical, thereby imposing significant scalability constraints on the broader application of this methodology.

## 6.2 Static Analysis for Vulnerable Version Identification

Static analysis approaches primarily comprise version-controlled commit tracing and code similarity matching. The former identifies vulnerability-inducing commits through backward traversal from fix commits, designating intermediate versions between inducing and fix commits as vulnerable. Typical algorithms include the SZZ [39] algorithm and its variants AG-SZZ [24], MA-SZZ [14], RA-SZZ [32], and V-SZZ [9]. Sliwerski et al. [39] proposed a technique known as Sliwerski-Zimmermann-Zeller

(SZZ for short) that uses the diff command provided by Version Control Systems to identify lines of code modified by changes that fixed bugs. By tracing code history, the SZZ technique uses the annotate command in VCS to identify changes that introduced errors. Many researchers have improved SZZ to enhance the accuracy of identifying error-inducing commits. Kim et al. [24] proposed the AG-SZZ method, which uses annotation graphs to obtain more precise results, addressing the deficiency of non-semantic lines in the original SZZ algorithm. Costa et al. [14] introduced the MA-SZZ method, which excludes meta-changes from all candidate bug-inducing commits, i.e., commits that do not change the source code. To further reduce false positives, Neto et al. proposed two versions of RA-SZZ [32], based on RefDiff [38] and Refactoring Miner [44], which consider the impact of refactored lines of code in the original SZZ algorithm. Bao et al. [9] proposed the V-SZZ method, which leverages line mapping algorithms to identify the earliest commit that modified the vulnerable lines and then considers these commits to be the vulnerability-inducing commits. Rosa et al. [37] used an oracle to evaluate the performance of existing SZZ and its variants.

Many works also identify vulnerable versions based on code similarity matching. ReDebug [21] uses a fast, syntax-based method that can scale to code bases the size of operating system distributions written in many different languages. MVP [46] designed a novel slicing method based on patch code, extracting vulnerability and repair signatures from vulnerable functions and their patched counterparts. VUDDY [23] reduces the number of signature comparisons by utilizing function-level granularity and length filtering techniques, proposing a scalable method to detect vulnerable code clones. VCCFinder [34], based on data collected from code repositories, trained an SVM classifier to tag suspicious commits. Similarly, VulPecker [28], based on a collected database of vulnerability patches and instances of vulnerable code, as well as an automatic selection of code similarity algorithms effective for specific vulnerabilities, solved the problem of source code vulnerability detection. VulDeePecker [49] improved the accuracy of VulPecker through deep learning. All these efforts target the C/C++ ecosystem, while VERJava [42] targets the Java ecosystem, which employs a two-stage approach to systematically assess vulnerable versions.

The emergence of LLMs has brought new solutions for security research. For example, Li et al. introduce IRIS [27], a neuro-symbolic approach combining LLMs with static analysis. The LLM infers taint-flow specifications for a project and guides vulnerability detection. Fakhri et al. present LLM4CVE [18], an iterative pipeline that uses LLMs to automatically generate patches for vulnerable functions. In terms of identifying vulnerable versions, Tang et al. propose LLM4SZZ [43] to enhance the classic SZZ algorithm by using an LLM to identify bug-inducing commits. However, vulnerabilities differ from bugs. Without vulnerability specific context and accurate tracing of inducing commits, LLM4SZZ falls short on vulnerable version identification. Our approach is specifically designed to address this issue.

## 7 Conclusion

In this paper, we presented *CaVulner*, a novel approach for the precise identification of vulnerable versions in Java open source projects. *CaVulner* integrates multiple sources of information, including method level change patterns, CVE and CWE metadata, and patch diffs, with the semantic reasoning power of large language models. It localizes vulnerable code blocks, traces the commits that introduced those vulnerabilities and derives an accurate set of affected version tags by deduplicating hunk hashes and normalizing version labels. Our evaluation on 176 real-world CVEs from 43 projects covering 8,730 versions demonstrates that *CaVulner* outperforms state-of-the-art approaches, including V-SZZ, VERJava, and LLM4SZZ in accuracy, precision, recall, and F1-score. While this study focuses on the Java ecosystem, we will adapt *CaVulner* to additional language ecosystems such as C, C++, and Python in future work.

## References

- [1] 2026. CaVulner. <https://sites.google.com/view/cavulner> [Online; accessed 2026-01-24].
- [2] GitHub, Inc. 2026. GitHub Advisory Database. <https://github.com/advisories>.
- [3] LangChain. 2026. LangChain. <https://www.langchain.com>.
- [4] Sonatype. 2026. Maven Central Repository. <https://search.maven.org>.
- [5] Nikolaos Alexopoulos, Manuel Brack, Jan Philipp Wagner, Tim Grube, and Max Mühlhäuser. 2022. How Long Do Vulnerabilities Live in the Code? A Large-Scale Empirical Measurement Study on FOSS Vulnerability Lifetimes. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, Boston, MA, 359–376. <https://www.usenix.org/conference/usenixsecurity22/presentation/alexopoulos>
- [6] Anna Ribeiro. 2025. WEF sounds alarm on software supply chain vulnerabilities, flags risks in open-source and third-party dependencies. <https://industrialcyber.co/supply-chain-security/wef-sounds-alarm-on-software-supply-chain-vulnerabilities-flags-risks-in-open-source-and-third-party-dependencies>.
- [7] Apache Software Foundation. 2026. struts. <https://github.com/apache/struts>.
- [8] Apache Software Foundation. 2026. tomcat. <https://github.com/apache/tomcat>.
- [9] Lingfeng Bao, Xin Xia, Ahmed E. Hassan, and Xiaohu Yang. 2022. V-SZZ: automatic identification of version ranges affected by CVE vulnerabilities. In *Proceedings of the 44th International Conference on Software Engineering (Pittsburgh, Pennsylvania) (ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 2352–2364. doi:10.1145/3510003.3510113
- [10] Black Duck Software, Inc. 2025. 2025 Open Source Security and Risk Analysis. <https://www.blackduck.com/resources/analyst-reports/open-source-security-risk-analysis.html>.
- [11] Marcel Böhme, Van-Thuan Pham, Manh-Dung Nguyen, and Abhik Roychoudhury. 2017. Directed Greybox Fuzzing. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (Dallas, Texas, USA) (CCS '17)*. Association for Computing Machinery, New York, NY, USA, 2329–2344. doi:10.1145/3133956.3134020
- [12] Hongxu Chen, Yinxing Xue, Yuekang Li, Bihuan Chen, Xiaofei Xie, Xiuheng Wu, and Yang Liu. 2018. Hawkeye: Towards a Desired Directed Grey-box Fuzzer. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (Toronto, Canada) (CCS '18)*. Association for Computing Machinery, New York, NY, USA, 2095–2108. doi:10.1145/3243734.3243849
- [13] Xingchu Chen, Chengwei Liu, Jialun Cao, Yang Xiao, Xinyue Cai, Yeting Li, Jingyi Shi, Tianqi Sun, Haiming Chen, and Wei Huo. 2025. Vulnerability-Affected Versions Identification: How Far Are We? *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE) (2025)*, 2970–2982.
- [14] Daniel Alencar da Costa, Shane McIntosh, Weiyi Shang, Uirá Kulesza, Roberta Coelho, and Ahmed E. Hassan. 2017. A Framework for Evaluating the Results of the SZZ Approach for Identifying Bug-Introducing Changes. *IEEE Transactions on Software Engineering* 43, 7 (2017), 641–657. doi:10.1109/TSE.2016.2616306
- [15] John Doll, Carson McCarthy, Hannah McDougall, and Suman Bhunia. 2025. Unraveling Log4Shell: Analyzing the Impact and Response to the Log4j Vulnerability. arXiv:2501.17760 [cs.CR] <https://arxiv.org/abs/2501.17760>
- [16] Ying Dong, Wenbo Guo, Yueqi Chen, Xinyu Xing, Yuqing Zhang, and Gang Wang. 2019. Towards the Detection of Inconsistencies in Public Security Vulnerability Reports. In *28th USENIX Security Symposium (USENIX Security 19)*. USENIX Association, Santa Clara, CA, 869–885. <https://www.usenix.org/conference/usenixsecurity19/presentation/dong>
- [17] Zhengjie Du, Yuekang Li, Yang Liu, and Bing Mao. 2022. WindRanger: a directed greybox fuzzer driven by deviation basic blocks. In *Proceedings of the 44th International Conference on Software Engineering (Pittsburgh, Pennsylvania) (ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 2440–2451. doi:10.1145/3510003.3510197
- [18] Mohamad Fakihi, Rahul Dharmaji, Halima Bouzidi, Gustavo Quiros Araya, Oluwatosin Ogundare, and Mohammad Abdullah Al Faruque. 2025. LLM4CVE: Enabling Iterative Automated Vulnerability Repair with Large Language Models. arXiv:2501.03446 [cs.SE] <https://arxiv.org/abs/2501.03446>
- [19] FasterXML. 2026. jackson-databind. <https://github.com/FasterXML/jackson-databind>.
- [20] Heqing Huang, Yiyuan Guo, Qingkai Shi, Peisen Yao, Rongxin Wu, and Charles Zhang. 2022. BEACON: Directed Grey-Box Fuzzing with Provable Path Pruning. In *2022 IEEE Symposium on Security and Privacy (SP)*. 36–50. doi:10.1109/SP46214.2022.9833751
- [21] Jiyong Jang, Abeer Agrawal, and David Brumley. 2012. ReDeBug: Finding Unpatched Code Clones in Entire OS Distributions. In *Proceedings of the 2012 IEEE Symposium on Security and Privacy (SP '12)*. IEEE Computer Society, USA, 48–62. doi:10.1109/SP.2012.13
- [22] Jenkins. 2026. jenkins. <https://github.com/jenkinsci/jenkins>.
- [23] Seulbae Kim, Seunghoon Woo, Heejo Lee, and Hakjoo Oh. 2017. VUDDY: A Scalable Approach for Vulnerable Code Clone Discovery. In *2017 IEEE Symposium on Security and Privacy (SP)*. 595–614. doi:10.1109/SP.2017.62
- [24] Sunghun Kim, Thomas Zimmermann, Kai Pan, and E. James Jr. Whitehead. 2006. Automatic Identification of Bug-Introducing Changes. In *21st IEEE/ACM International Conference on Automated Software Engineering (ASE'06)*. 81–90.

- doi:10.1109/ASE.2006.23
- [25] Klaus Krippendorff. 2019. *Content Analysis: An Introduction to Its Methodology* (4th ed.). SAGE Publications, Inc., Thousand Oaks, CA. doi:10.4135/9781071878781
  - [26] Siyuan Li, Yuekang Li, Zuxin Chen, Chaopeng Dong, Yongpan Wang, Hong Li, Yongle Chen, and Hongsong Zhu. 2025. TransferFuzz: Fuzzing with Historical Trace for Verifying Propagated Vulnerability Code. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. 268–280. doi:10.1109/ICSE55347.2025.00061
  - [27] Ziyang Li, Saikat Dutta, and Mayur Naik. 2025. IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=9LdJDU7E91>
  - [28] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Hanchao Qi, and Jie Hu. 2016. VulPecker: an automated vulnerability detection system based on code similarity analysis. In *Proceedings of the 32nd Annual Conference on Computer Security Applications* (Los Angeles, California, USA) (ACSAC '16). Association for Computing Machinery, New York, NY, USA, 201–213. doi:10.1145/2991079.2991102
  - [29] Liferay. 2026. liferay-portal. <https://github.com/liferay/liferay-portal>.
  - [30] Changhua Luo, Wei Meng, and Penghui Li. 2023. SelectFuzz: Efficient Directed Fuzzing with Selective Path Exploration. In *2023 IEEE Symposium on Security and Privacy (SP)*. 2693–2707. doi:10.1109/SP46215.2023.10179296
  - [31] National Institute of Standards and Technology. 2026. National Vulnerability Database. <https://nvd.nist.gov>.
  - [32] Edmilson Campos Neto, Daniel Alencar da Costa, and Uirá Kulesza. 2018. The impact of refactoring changes on the SZZ algorithm: An empirical study. In *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 380–390. doi:10.1109/SANER.2018.8330225
  - [33] Viet Hung Nguyen, Stanislav Dashevskiy, and Fabio Massacci. 2016. An automatic method for assessing the versions affected by a vulnerability. In *Empirical Software Engineering*. 2268–2297.
  - [34] Henning Perl, Sergej Dechand, Matthew Smith, Daniel Arp, Fabian Yamaguchi, Konrad Rieck, Sascha Fahl, and Yasemin Acar. 2015. VCCFinder: Finding Potential Vulnerabilities in Open-Source Projects to Assist Code Audits. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security* (Denver, Colorado, USA) (CCS '15). Association for Computing Machinery, New York, NY, USA, 426–437. doi:10.1145/2810103.2813604
  - [35] Serena Elisa Ponta, Henrik Plate, Antonino Sabetta, Michele Bezzi, and Cédric Dangremont. 2019. A Manually-Curated Dataset of Fixes to Vulnerabilities of Open-Source Software. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. 383–387. doi:10.1109/MSR.2019.00064
  - [36] Giovanni Rosa, Luca Pascarella, Simone Scalabrino, Rosalia Tufano, Gabriele Bavota, Michele Lanza, and Rocco Oliveto. 2021. Evaluating SZZ Implementations Through a Developer-Informed Oracle. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. 436–447. doi:10.1109/ICSE43902.2021.00049
  - [37] Giovanni Rosa, Luca Pascarella, Simone Scalabrino, Rosalia Tufano, Gabriele Bavota, Michele Lanza, and Rocco Oliveto. 2023. A comprehensive evaluation of SZZ Variants through a developer-informed oracle. *J. Syst. Softw.* 202, C (Aug. 2023), 19 pages. doi:10.1016/j.jss.2023.111729
  - [38] Danilo Silva and Marco Tulio Valente. 2017. RefDiff: detecting refactorings in version histories. In *Proceedings of the 14th International Conference on Mining Software Repositories* (Buenos Aires, Argentina) (MSR '17). IEEE Press, 269–279. doi:10.1109/MSR.2017.14
  - [39] Jacek Sliwerski, Thomas Zimmermann, and Andreas Zeller. 2005. When do changes induce fixes?. In *Proceedings of the 2005 International Workshop on Mining Software Repositories* (St. Louis, Missouri) (MSR '05). Association for Computing Machinery, New York, NY, USA, 1–5. doi:10.1145/1083142.1083147
  - [40] Spring Projects. 2026. spring-framework. <https://github.com/spring-projects/spring-framework>.
  - [41] Spring Projects. 2026. spring-security. <https://github.com/spring-projects/spring-security>.
  - [42] Qing Sun, Lili Xu, Yang Xiao, Feng Li, He Su, Yiming Liu, Hongyun Huang, and Wei Huo. 2022. VERJava: Vulnerable Version Identification for Java OSS with a Two-Stage Analysis. In *2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 329–339. doi:10.1109/ICSME55016.2022.00037
  - [43] Lingxiao Tang, Jiakun Liu, Zhongxin Liu, Xiaohu Yang, and Lingfeng Bao. 2025. LLM4SZZ: Enhancing SZZ Algorithm with Context-Enhanced Assessment on Large Language Models. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA016 (June 2025), 23 pages. doi:10.1145/3728885
  - [44] Nikolaos Tsantalos, Matin Mansouri, Laleh M. Eshkevari, Davood Mazinanian, and Danny Dig. 2018. Accurate and efficient refactoring detection in commit history. In *Proceedings of the 40th International Conference on Software Engineering* (Gothenburg, Sweden) (ICSE '18). Association for Computing Machinery, New York, NY, USA, 483–494. doi:10.1145/3180155.3180206
  - [45] Junjie Wang, Yuhua Ma, Xiaofei Xie, Xiaoning Du, and Xiangwei Zhang. 2025. PatchFuzz: Patch Fuzzing for JavaScript Engines. arXiv:2505.00289 [cs.CR] <https://arxiv.org/abs/2505.00289>
  - [46] Yang Xiao, Bihuan Chen, Chendong Yu, Zhengzi Xu, Zimu Yuan, Feng Li, Binghong Liu, Yang Liu, Wei Huo, Wei Zou, and Wenchang Shi. 2020. MVP: Detecting Vulnerabilities using Patch-Enhanced Vulnerability Signatures. In *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, 1165–1182. <https://www.usenix.org/>

conference/usenixsecurity20/presentation/xiao

[47] Wei You, Peiyuan Zong, Kai Chen, XiaoFeng Wang, Xiaojing Liao, Pan Bian, and Bin Liang. 2017. SemFuzz: Semantics-based Automatic Generation of Proof-of-Concept Exploits. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security* (Dallas, Texas, USA) (CCS '17). Association for Computing Machinery, New York, NY, USA, 2139–2154. doi:10.1145/3133956.3134085

[48] Jia-Ming Zhang, Zhan-Qi Cui, Xiang Chen, Huan-Huan Wu, Li-Wei Zheng, and Jian-Bin Liu. 2022. DeltaFuzz: Historical Version Information Guided Fuzz Testing. *J. Comput. Sci. Technol.* 37, 1 (Feb. 2022), 29–49. doi:10.1007/s11390-021-1663-7

[49] Deqing Zou, Sujuan Wang, Shouhuai Xu, Zhen Li, and Hai Jin. 2021. VulDeePecker: A Deep Learning-Based System for Multiclass Vulnerability Detection. *IEEE Transactions on Dependable and Secure Computing* 18, 5 (2021), 2224–2236. doi:10.1109/TDSC.2019.2942930